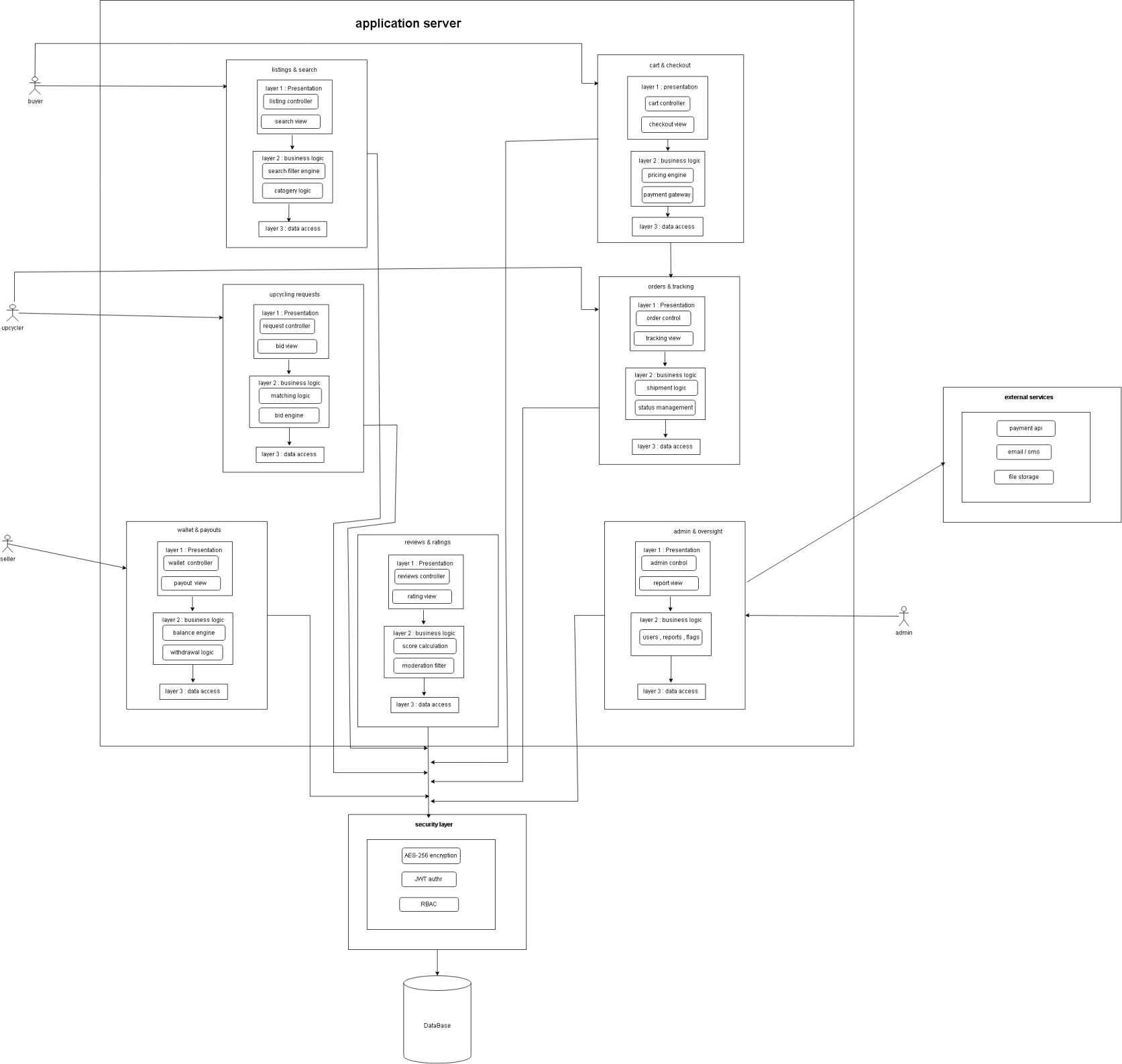
application server
buyer
listings & search
layer 1 : Presentation
listing controller
search view
layer 2 : business logic
search filter engine
catogery logic
layer 3 : data access
cart & checkout
layer 1 : presentation
cart controller
checkout view
layer 2 : business logic
pricing engine
payment gateway
layer 3 : data access
upcycler
upcycling requests
layer 1 : Presentation
request controller
bid view
layer 2 : business logic
matching logic
bid engine
layer 3 : data access
orders & tracking
layer 1 : Presentation
order control
tracking view
layer 2 : business logic
shipment logic
status management
layer 3 : data access
external services
payment api
email / sms
file storage
seller
wallet & payouts
layer 1 : Presentation
wallet controller
payout view
layer 2 : business logic
balance engine
withdrawal logic
layer 3 : data access
reviews & ratings
layer 1 : Presentation
reviews controller
rating view
layer 2 : business logic
score calculation
moderation filter
layer 3 : data access
admin & oversight
layer 1 : Presentation
admin control
report view
layer 2 : business logic
users , reports , flags
layer 3 : data access
admin
security layer
AES-256 encryption
JWT authr
RBAC
DataBase

Upcycling Platform System Architecture

The previous diagram illustrates a high-level overview of the system's architecture, showcasing the interaction between different modules, layers, and external services within a unified framework. It highlights how the system manages data flow from user interfaces down to secure persistent storage.

Primary Architectural Design Patterns:

The system's architectural integrity is built upon three foundational patterns, chosen specifically to balance scalability, simplicity, and long-term maintainability.

1. Architectural Style: Modular Monolith

Why we chose it:

The system is designed as a Modular Monolith where core domains (Users, Items, Upcycling Process, Orders, Payments) are separated into independent internal modules within a single deployable application.

Why not other architectural styles?

Other architectural styles such as Microservices, Event-Driven Architecture, and Serverless were not chosen because they introduce unnecessary complexity, distributed system overhead, and higher operational cost compared to the current system needs. Microservices require service orchestration, inter-service communication, and complex deployment pipelines. Event-Driven systems add additional complexity in managing asynchronous flows and event consistency. Serverless architectures, while scalable, introduce cold start issues and vendor dependency.

Since the system is still in its early-to-mid development stage and focuses on core functionality (users, items, upcycling workflow, and orders), a Modular Monolith provides the best balance between simplicity and structure. It allows all modules to exist within a single deployable system while still maintaining clear separation of concerns, making development, testing, and maintenance significantly easier.

2. Communication Pattern: Client-Server

Why we chose it:

The system follows a Client-Server architecture where the Client (Web/Mobile Application) interacts with a centralized Server (Backend System responsible for business logic and data processing).

Why not Peer-to-Peer (P2P)?

P2P is not suitable because the system requires centralized control over users, items, transactions, and upcycling workflows. A server-based model ensures data consistency, secure authentication, and proper transaction handling.

3. Internal Pattern: Layered (Three-Tier) Architecture

Why we chose it:

The system is structured into three layers: Presentation Layer, Business Logic Layer, and Data Access Layer. This ensures clear separation of concerns and improves maintainability and scalability.

Why not a Single-Layer (Flat) Pattern?

A flat architecture would tightly couple UI, logic, and database operations, making the system hard to scale and maintain. Layered architecture allows independent updates to each layer without affecting the rest of the system.